

PATRÓN DE DISEÑO

DECORATOR + FACADE

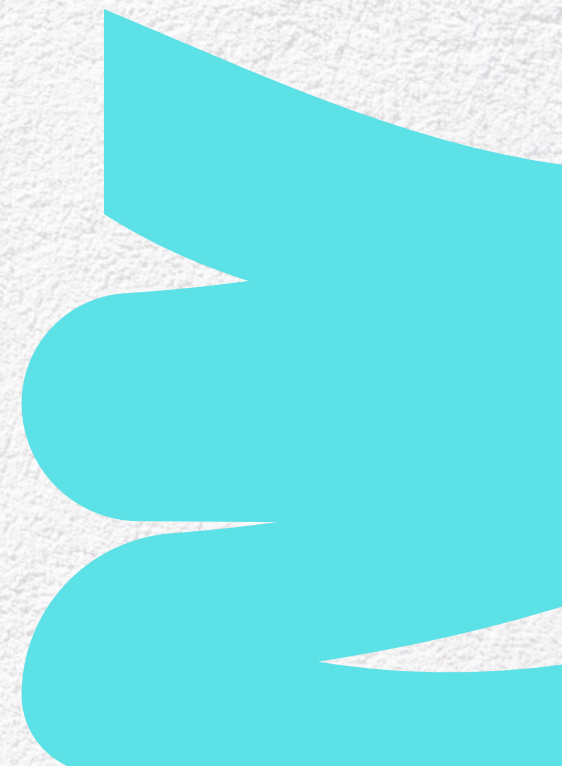
Mariana Botero Gomez



PATRONES DE DISEÑO ESTRUCTURALES



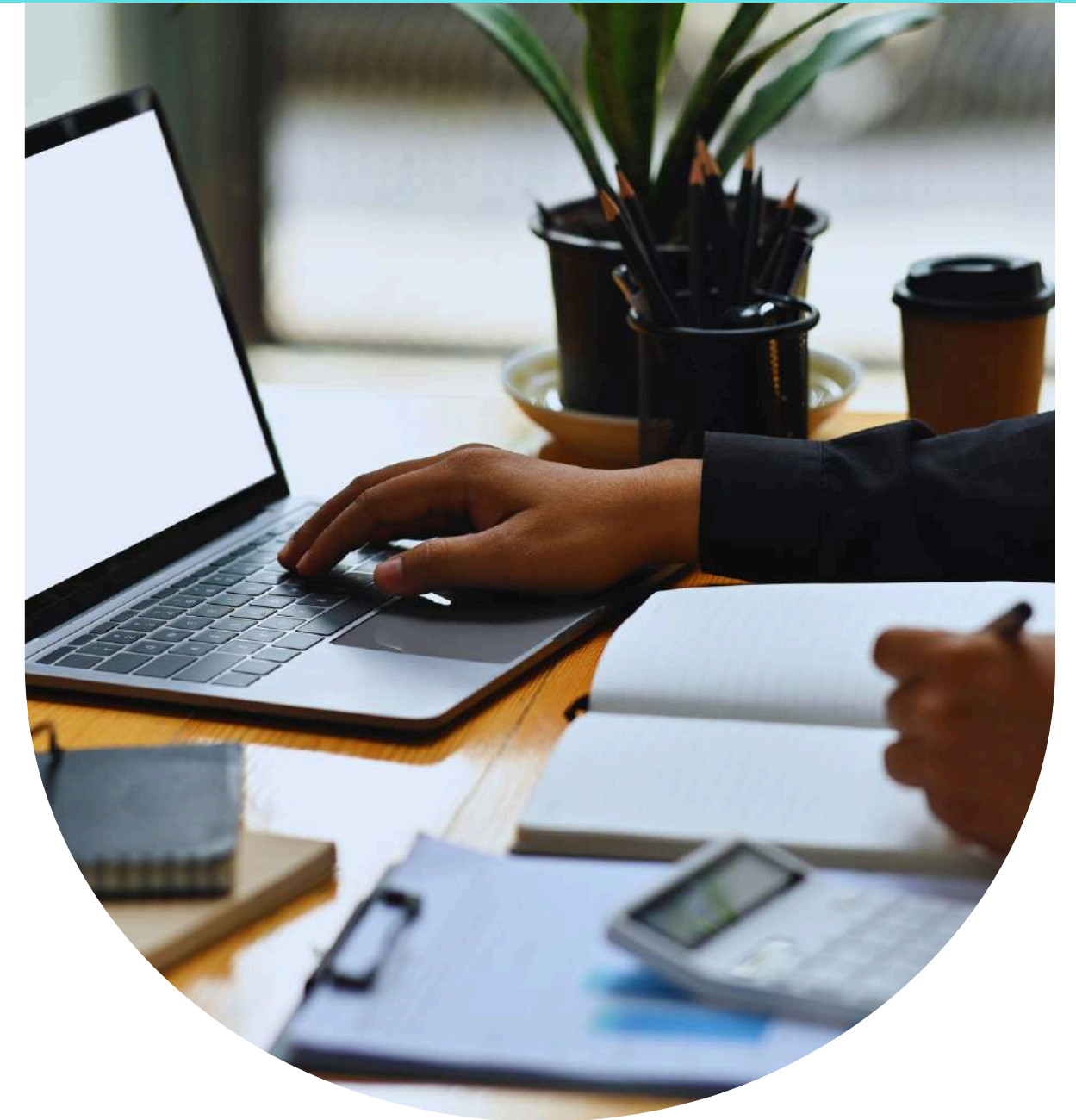
Explican cómo ensamblar objetos y clases en estructuras más grandes y flexibles.
Entre ellos están el Facade y el Decorator



DECORATOR

También conocido como wrapper permite agregarle responsabilidades a un objeto envolviéndolo dentro de otro objeto, sin tocar su clase original y sin usar herencia.

Por ejemplo, una cafetería
-Tenemos un cafe base al que le podemos agregar leche, caramelo, canela



PARA QUE SIRVE

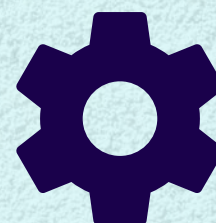
1

Extender
comportamiento



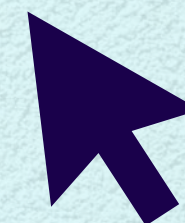
2

Evita la herencia
excesiva



3

Principio
abierto/cerrado





COMPONENTES PRINCIPALES

icomponent

Interfaz común para los objetos que pueden recibir responsabilidades adicionales.

concreteComponent

La clase principal o básica que define el objeto al que se le añadirá la funcionalidad.

componentDecorator

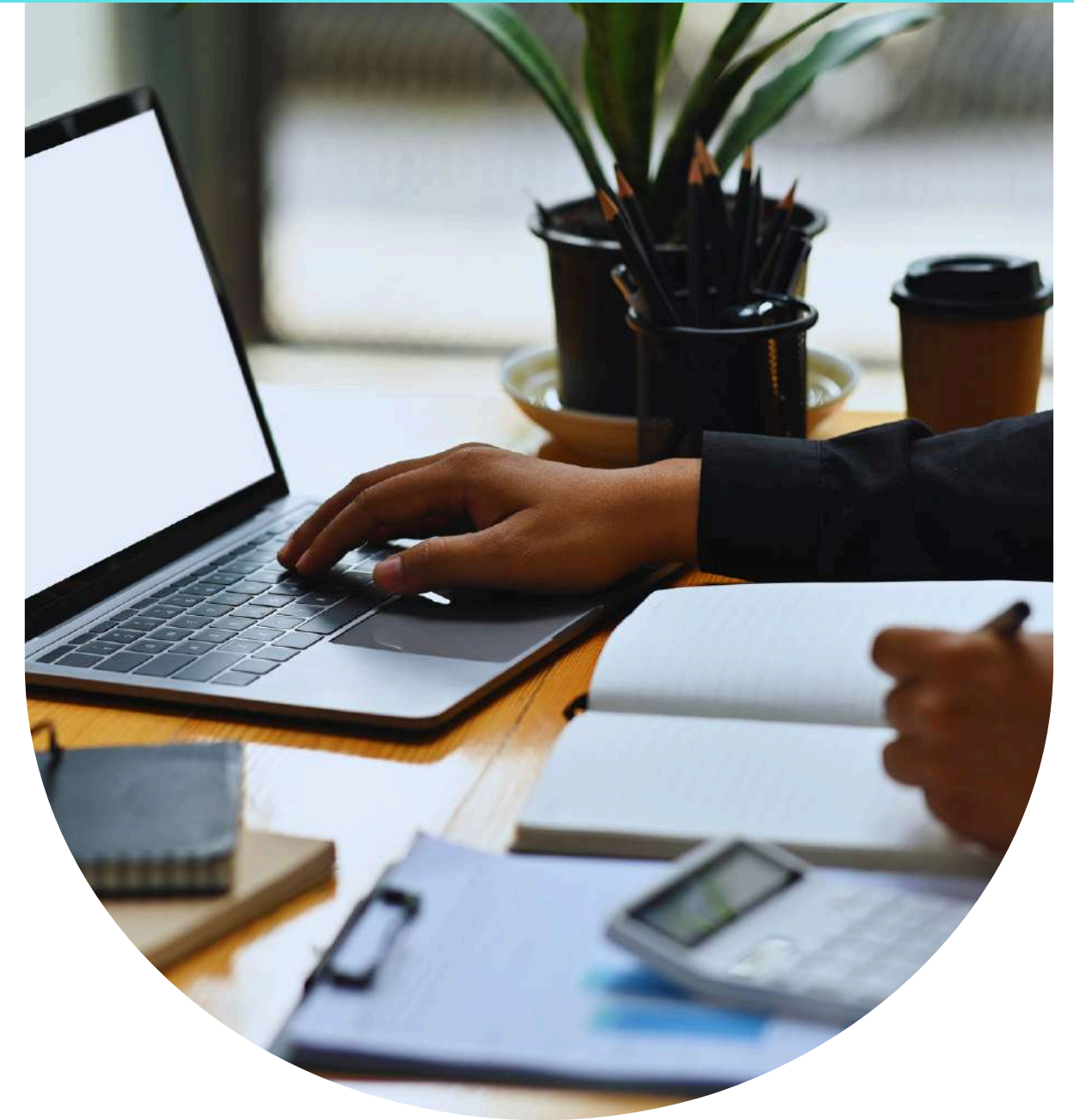
Clase abstracta que implementa la misma interfaz del componente y contiene una referencia al objeto envuelto.

FACADE

es un patrón estructural que ofrece una interfaz simplificada y unificada para ocultar la complejidad de un subsistema o un conjunto de clases.

Por ejemplo, una cafetería

- Tenemos un cafe base al que le podemos agregar leche, caramelo, canela



PARA QUE SIRVE

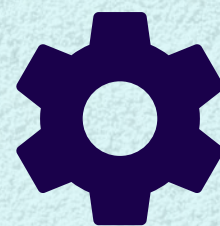
1

Ocultar la
complejidad



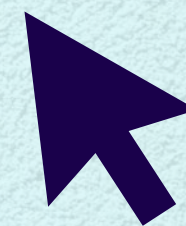
2

Reducir el
acoplamiento



3

Mejorar la
complejidad





COMPONENTES PRINCIPALES

Facade

Conoce qué clases del subsistema son responsables de una solicitud y delega las peticiones de los clientes a los objetos adecuados del subsistema.

Client

Es el código, aplicación o usuario que necesita realizar una tarea interactuando con el sistema.

Subsystem classes

Son el conjunto de clases complejas, librerías de terceros o módulos que realizan el trabajo real de la aplicación.



EJEMPLO

El usuario tendría que encender el proyector, bajar la pantalla, encender el amplificador, ajustar el volumen, configurar la entrada de video y encender el reproductor de Blu-ray (seis pasos distintos).

Imaginamos un sistema de cine en casa.

Subsistemas complejos:

```
public class Proyector
{
    public void Encender() => Console.WriteLine("Proyector encendido.");
    public void Apagar() => Console.WriteLine("Proyector apagado.");
}
```



GRACIAS
POR SU ATENCIÓN